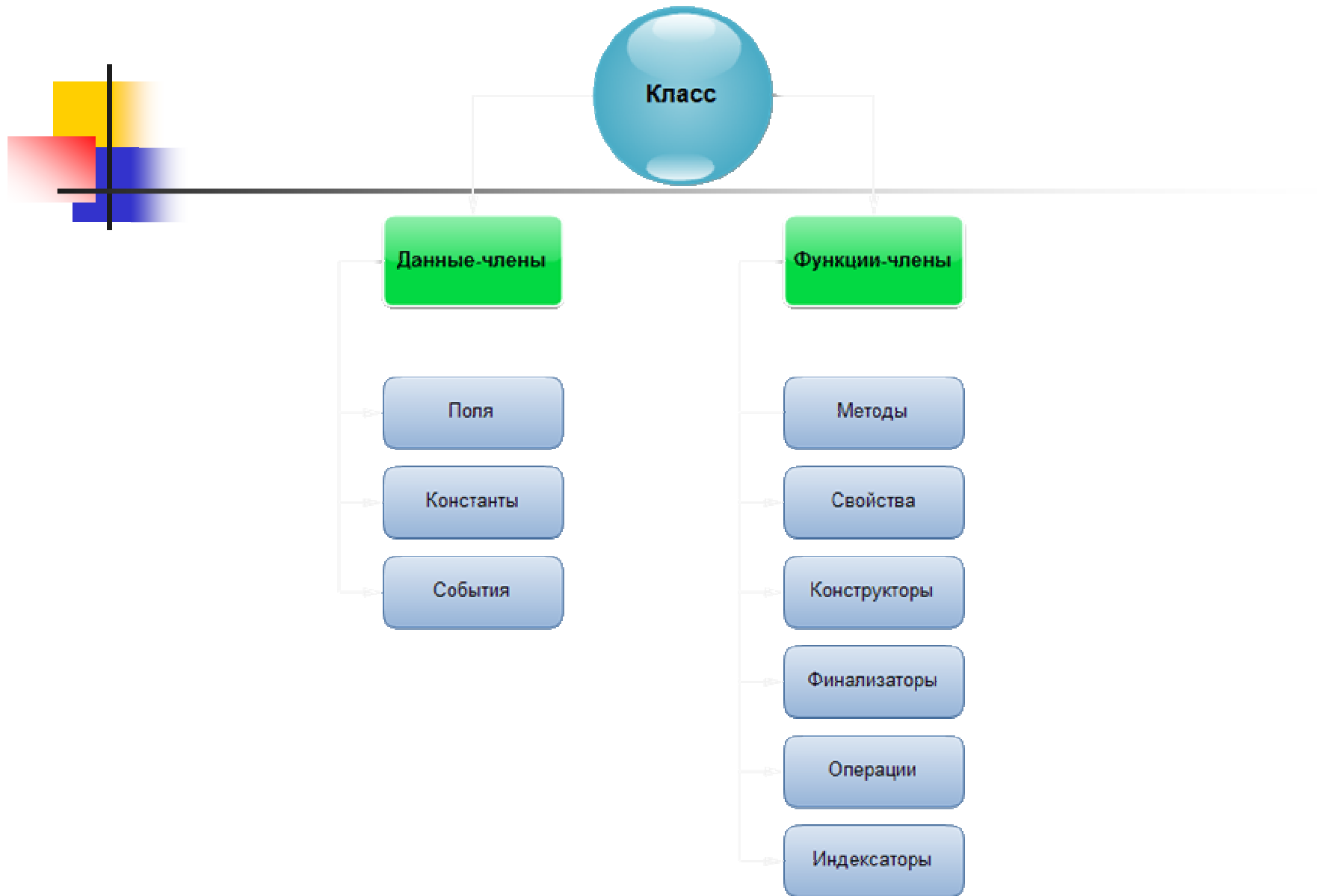
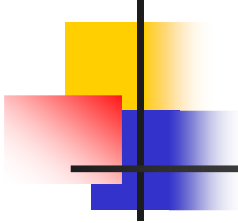




C# и .Net

Лекция 2 Классы

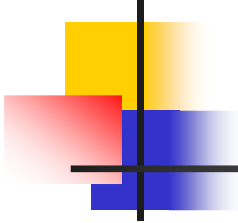




Модификаторы доступа для классов

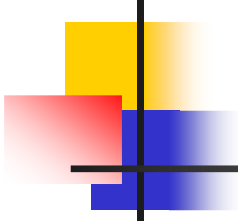
```
[модификатор доступа] class [имя_класса]
{
    //тело класса
}
```

- *public* – доступ к классу возможен из любого места одной сборки, либо из другой сборки, на которую есть ссылка;
- *internal* – доступ к классу возможен только из сборки, в которой он объявлен.



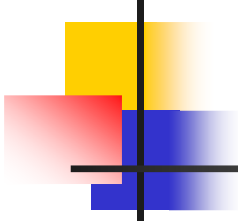
Объявление методов

```
[модификаторы] тип_возврата ИмяМетода([параметры])  
{  
    // Тело метода  
}
```



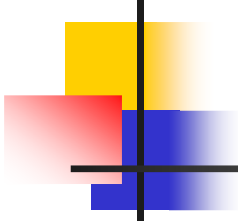
Уровни доступа для членов класса

- *public* – доступ к члену возможен из любого места одной сборки, либо из другой сборки, на которую есть ссылка;
- *protected* – доступ к члену возможен только внутри класса, либо в классе-наследнике (при наследовании);
- *private* – доступ к члену возможен только внутри класса;
- *internal* – доступ к члену возможен только из сборки, в которой он объявлен;
- *protected internal* - доступ к члену возможен из одной сборки, либо из класса-наследника другой сборки.



Создание объектов

```
Building house; // объявить ссылку на объект  
house = new Building();  
// распределить память для объекта типа Building
```



Пример объявлений

```
class Student
{
    private string firstName;
    private string lastName;
    private int age;
    public string group;
    // не рекомендуется использовать public для поля
}
```

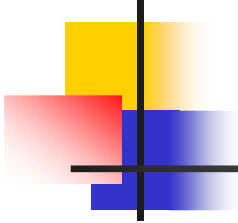


Пример создания объекта

```
static void Main(string[] args)
{
    Student student1 = new Student();

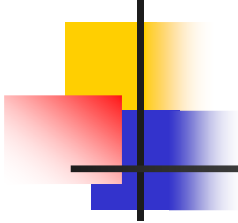
    student1.group = "Group1";
    student1.firtsName = "Nikolay";
    //Ошибка, нет доступа к полю firtsName.
    // Программа не скомпилируется

    Console.WriteLine(student1.group);
    // выводит на экран "Group1"
}
```

Вложенные классы

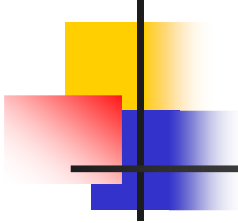
```
public class OuterClass {  
    protected class InnerClass {  
        // и т.д.  
    } // и т.д.  
}
```



Конструкторы и деструкторы

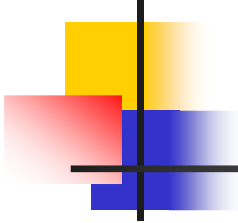
```
доступ имя_класса(список_параметров)
{
    // тело конструктора
}
```

```
~имя_класса()
{
    // код деструктора
}
```



Пример

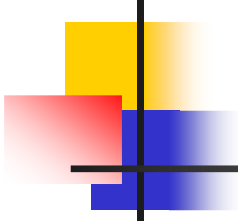
```
class Destruct {  
    public int x;  
    public Destruct(int i) {  
        x = i;  
    }  
    // Вызывается при утилизации объекта.  
    ~Destruct() {  
        Console.WriteLine("Уничтожить " + x);  
    }  
    // Создает объект и тут же уничтожает его.  
    public void Generator(int i) {  
        Destruct o = new Destruct (i);  
    }  
}
```



Пример

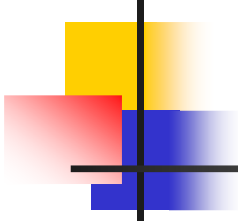
```
class Destruct {
    public int x;
    public Destruct(int i) { x = i; }
    ~Destruct() { Console.WriteLine("Уничтожить " + x); }
    public void Generator(int i) { Destruct o = new Destruct (i); }
}

class Program {
    static void Main(string[] args) {
        int count; Destruct ob = new Destruct (0);
        for (count=1; count < 100000; count++)
            ob.Generator(count);
        Console.WriteLine("Готово!");
    }
}
```



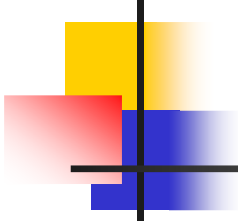
Ключевое слово this

```
class Car {  
    private double mileage;  
    private double fuel;  
  
    public Car(double mileage, double fuel) {  
        this.mileage = mileage;  
        this.fuel = fuel;  
    }  
}  
class Program {  
    static void Main(string[] args) {  
        Car newCar = new Car(100, 50);  
    }  
}
```



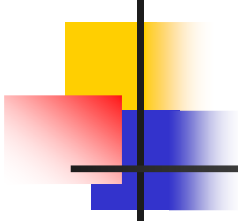
Пример использования нескольких конструкторов

```
class Car {
    private double mileage; private double fuel;
    public Car() {
        mileage = 0; fuel = 0;
    }
    public Car(double mileage, double fuel) {
        this.mileage = mileage; this.fuel = fuel;
    }
}
class Program {
    static void Main(string[] args) {
        Car newCar = new Car();
        Car newCar2 = new Car(100, 50);
        Car newCar2 = new Car();
    }
}
// ошибка, в классе не определен конструктор без параметров
```



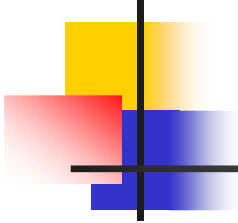
Свойства

```
class SimpProp {  
    int prop;  
    public SimpProp() { prop = 0; }  
    public int MyProp {  
        get {  
            return prop;  
        }  
        set {  
            if (value >= 0) prop = value;  
        }  
    }  
}
```



Заккрытие свойства на запись

```
class Student {  
    private int year;  
  
    public Student(int y) {  
        year = y;  
    }  
  
    public int Year {  
        get {  
            return year;  
        }  
    }  
}
```

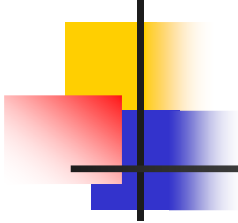



Автоматические свойства

```
class Student {  
    public int Year { get; set; }  
}
```

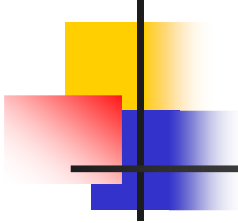
```
public int Year { private get; set; } // свойство только на запись  
public int Year { get; private set; } // свойство только на чтение
```

```
class Program {  
    static void Main(string[] args) {  
        Student st1 = new Student();  
  
        st1.Year = 0;  
        Console.WriteLine(st1.Year);  
        Console.ReadKey();  
    }  
}
```



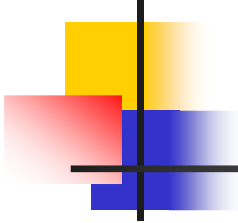
Индексаторы

```
public int this[int index] {  
    // get and set accessors  
}
```



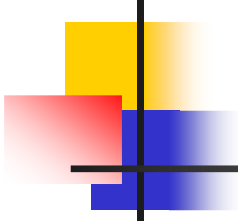
Пример

```
class TempRecord {  
    private float[] temps = new float[10] { 56.2F, 56.7F, 56.5F,  
        56.9F, 58.8F, 61.3F, 65.9F, 62.1F, 59.2F, 57.5F };  
    public int Length {  
        get { return temps.Length; }  
    }  
    public float this[int index] {  
        get { return temps[index]; }  
        set { temps[index] = value; }  
    }  
}
```



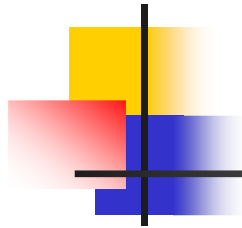
Пример

```
static void Main() {  
    TempRecord tempRecord = new TempRecord();  
    tempRecord[3] = 58.3F;  
    tempRecord[5] = 60.1F;  
    for (int i = 0; i < 10; i++) {  
        System.Console.WriteLine("Element #{0} = {1}", i,  
            tempRecord[i]);  
    }  
}
```



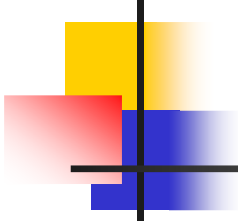
Пример

```
static void Main() {  
    TempRecord tempRecord = new TempRecord();  
    tempRecord[3] = 58.3F;  
    tempRecord[5] = 60.1F;  
    for (int i = 0; i < 10; i++) {  
        System.Console.WriteLine("Element #{0} = {1}", i,  
            tempRecord[i]);  
    }  
}
```



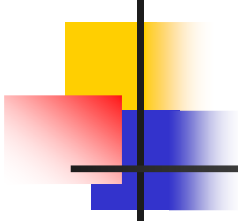
Пример 2

```
class DayCollection {  
    string[] days = { "Sun", "Mon", "Tues", "Wed", "Thurs", "Fri",  
        "Sat" };  
    private int GetDay(string testDay) {  
        for (int j = 0; j < days.Length; j++) {  
            if (days[j] == testDay) {  
                return j;  
            }  
        }  
        throw new System.ArgumentOutOfRangeException(testDay,  
            "testDay must be in the form \"Sun\", \"Mon\", etc");  
    }  
}
```



Пример 2

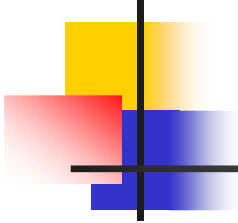
```
public int this[string day] {  
    get { return (GetDay(day)); }  
}  
}
```



Статические классы

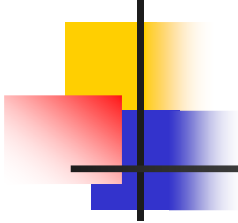
```
static class StaticUtilities {  
    public static void HelperMethod () { }  
}
```

```
StaticUtilities.HelperMethod();
```

Наследование

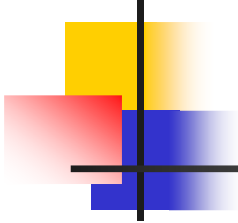
- наследование реализации
- наследование интерфейса



Наследование реализации

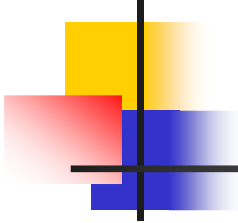
```
class УнаследованныйКласс: БазовыйКласс {  
    //Данные-члены и функции-члены  
}
```

```
public class MyDerivedClass: MyBaseClass, IInterfacel, IInterface2 {  
    // и т.д.  
}
```



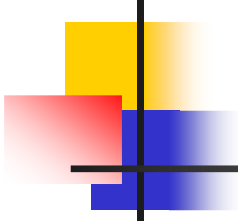
Для структур синтаксис выглядит так:

```
public struct MyDerivedStruct: IInterface1, IInterface2 {  
    // и т.д.  
}
```



Два фрагмента кода эквивалентны

```
class MyClass: Object // наследуется от System.Object
{
    // и т.д.
}
class MyClass // наследуется от System.Object
{
    // и т.д.
}
```

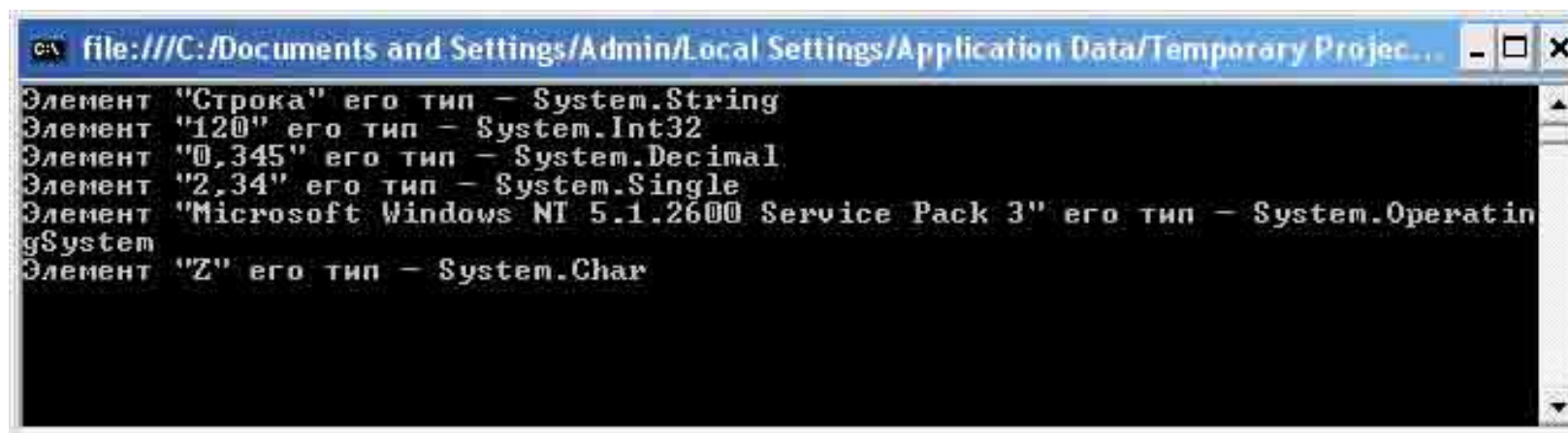


Массив типа object

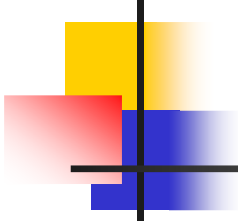
```
static void Main(string[] args) {  
    var myOS = Environment.OSVersion;  
    object[] myArr = { "Строка", 120, 0.345m, 2.34f, myOS, 'Z' };  
    foreach (object obj in myArr)  
        Console.WriteLine("Элемент \"{0}\" его тип - {1}", obj,  
                            obj.GetType());  
}
```



Результат программы

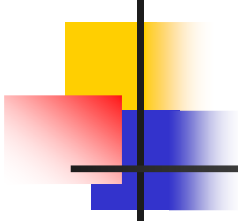


```
file:///C:/Documents and Settings/Admin/Local Settings/Application Data/Temporary Projec...
Элемент "Строка" его тип - System.String
Элемент "120" его тип - System.Int32
Элемент "0,345" его тип - System.Decimal
Элемент "2,34" его тип - System.Single
Элемент "Microsoft Windows NT 5.1.2600 Service Pack 3" его тип - System.Operationin
gSystem
Элемент "Z" его тип - System.Char
```



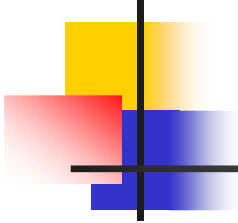
Вызов конструктора базового класса

```
[имя_конструктора_класса-наследника] ([аргументы]) : base  
    ([аргументы])  
    {  
        // тело конструктора  
    }
```



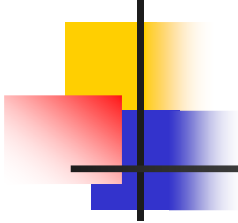
Пример

```
class Animal {  
    public string Name { get; set; }  
  
    public Animal(string name) {  
        Name = name;  
    }  
}  
class Perrot : Animal {  
    public double BeakLength { get; set; } // длина клюва  
  
    public Perrot(string name, double beak) : base(name) {  
        BeakLength = beak;  
    }  
}
```

Оператор is 1

```
class Animal {
    public string Name { get; set; }
    public Animal(string name) {
        Name = name;
    }
}
class Dog : Animal {
    public Dog(string name) : base(name) { }
    public void Guard() {
        // собака охраняет
    }
}
class Cat : Animal {
    public Cat(string name) : base(name) { }
    public void CatchMouse() {
        // кошка ловит мышь
    }
}
```



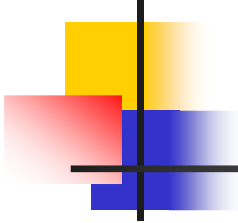
Оператор is 2

```
static void Main(string[] args) {  
    Dog dog1 = new Dog("Барбос");  
  
    Console.WriteLine(dog1 is Dog); // true  
    Console.WriteLine(dog1 is Animal); // true  
    Console.WriteLine(dog1 is Cat); // false  
    Console.ReadLine();  
}
```



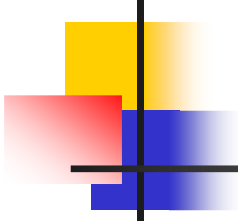
Оператор is 3

```
class Program {  
    static void Main(string[] args) {  
        List animals = new List();  
        animals.Add(new Dog("Барбос"));  
        animals.Add(new Cat("Барсик"));  
        animals.Add(new Dog("Палкан"));  
  
        foreach (Animal animal in animals) {  
            if (animal is Dog)  
                ((Dog)animal).Guard();  
            else  
                ((Cat)animal).CatchMouse();  
        }  
    }  
}
```



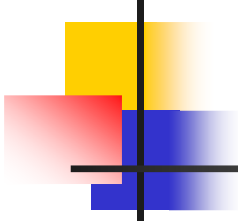
Оператор as

```
class Program {  
    static void Main(string[] args) {  
        List animals = new List();  
        animals.Add(new Dog("Барбос"));  
        animals.Add(new Cat("Барсик"));  
        animals.Add(new Dog("Палкан"));  
  
        foreach (Animal animal in animals) {  
            if (animal is Dog)  
                (animal as Dog).Guard();  
            else  
                (animal as Cat).CatchMouse();  
        }  
    }  
}
```



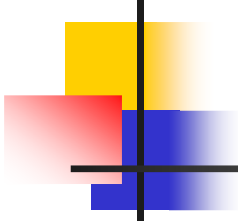
Виртуальные методы

```
class MyBaseClass {  
    public virtual string VirtualMethod () {  
        return "Это - виртуальный метод,  
        определенный в MyBaseClass";  
    }  
}
```



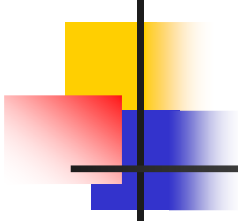
Виртуальные методы - переопределение

```
class MyDerivedClass: MyBaseClass {  
    public override string VirtualMethod() {  
        return "Этот переопределенный метод  
        объявлен в MyDerivedClass";  
    }  
}
```



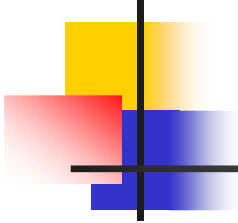
Виртуальные свойства

```
public virtual string ForeName {  
    get { return foreName; }  
    set { foreName = value; }  
}  
private string foreName;
```



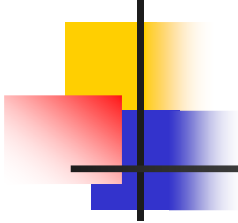
Вызов базовых версий функций

```
class CustomerAccount {  
    public virtual decimal CalculatePrice () {  
        return 30.0M;  
    }  
}  
  
class GoldAccount: CustomerAccount {  
    public override decimal CalculatePrice () {  
        return base.CalculatePrice () * 0.9M;  
    }  
}
```

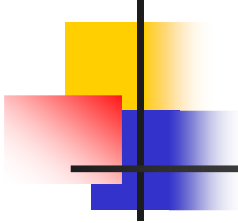
Абстрактные классы и функции

```
abstract class Building {  
    public abstract decimal CalculateHeatingCost ();  
}  
class MyBuilding : Building {  
    public override decimal CalculateHeatingCost() {  
        return 35.2M;  
    }  
}
```



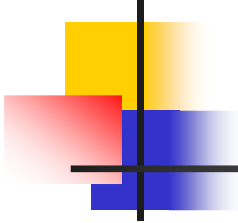
Запечатанные классы

```
sealed class FinalClass {  
    // и т.д.  
}  
class DerivedClass: FinalClass  
// Неверно. Ошибка компиляции.  
{  
    // и т.д.  
}
```



Запечатанные методы

```
class MyClass: MyClassBase {  
    public sealed override void FinalMethod() {  
        // и т.д.  
    }  
}  
  
class DerivedClass: MyClass {  
    public override void FinalMethod ()  
    // Неверно. Ошибка компиляции.  
    { // и т.д. }  
}
```



Методы System.Object

ToString()

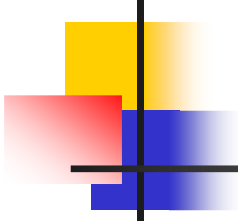
GetHashCode()

Equals() и ReferenceEquals()

Finalize()

GetType()

Clone()



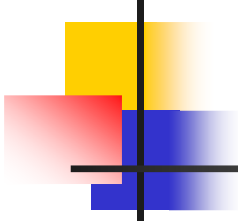
Упаковка и распаковка

```
int a=10;  
object b;  
b=a;
```

WriteLine(a.ToString) – неявная упаковка в object

a = b – это ошибка

a = (int)b; - распаковка



Сравнение объектов в C#.NET

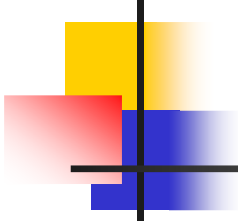
```
public static bool ReferenceEquals(object objA, object objB) {  
    return objA == objB;  
}
```

```
public static bool Equals(object objA, object objB) {  
    return objA == objB || (objA != null && objB != null &&  
        objA.Equals(objB)); }  

```

```
public virtual bool Equals(object obj);
```

```
public static bool operator == (Foo left, Foo right);
```

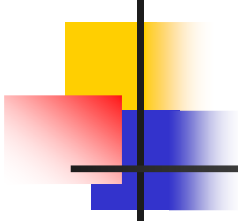


Сравнение двух строк этим методом ReferenceEquals

```
static void Main(string[] args) {  
    string a = "Hello";  
    string b = "Hello";  
    if(object.ReferenceEquals(a,b))  
        Console.WriteLine("Same objects");  
    else  
        Console.WriteLine("Not the same objects");  
}
```

Что в результате?

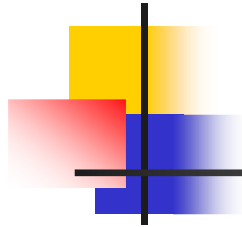
Интернирование строк



Перегрузка Equals(object obj)

1

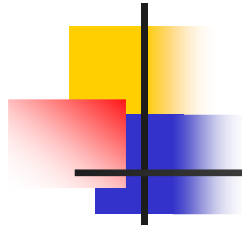
```
public class Person {  
    private string idNumber; private string personName;  
    public Person(string name, string id) {  
        this.personName = name; this.idNumber = id;  
    }  
}
```

Перегрузка Equals(object obj)

2

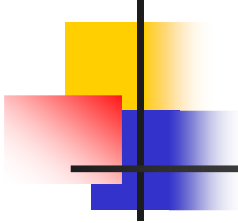
```
public override bool Equals(Object obj) {  
    Person personObj = obj as Person;  
    if (personObj == null)  
        return false;  
    else  
        return idNumber.Equals(personObj.idNumber);  
}  
public override int GetHashCode() {  
    return this.idNumber.GetHashCode();  
}  
}
```



Перегрузка Equals(object obj)

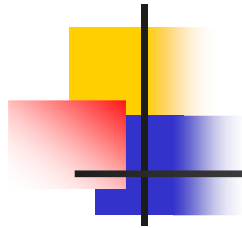
3

```
public class Example {  
    public static void Main() {  
        Person p1 = new Person("John", "63412895");  
        Person p2 = new Person("Jack", "63412895");  
        Console.WriteLine(p1.Equals(p2));  
        Console.WriteLine(Object.Equals(p1, p2));  
    }  
}  
  
// The example displays the following output:  
// True  
// True
```



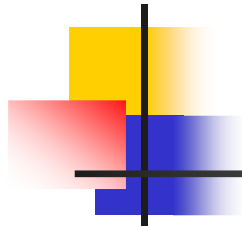
Рекомендации при переопределении Equals(Object)

- Типы, реализующие интерфейс Comparable, должны переопределять метод Equals(Object).
- Типы, которые переопределяют Equals(Object) также необходимо переопределить GetHashCode.
- Если перегружается оператор ==, необходимо также переопределить метод Equals(Object).



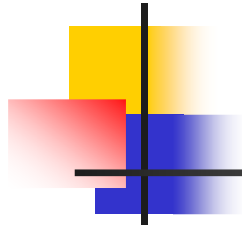
Пример class Point

```
class Point {  
    protected int x, y;  
    public Point() {  
        this.x = 0; this.y = 0;  
    }  
    public Point(int X, int Y) {  
        this.x = X; this.y = Y;  
    }  
}
```



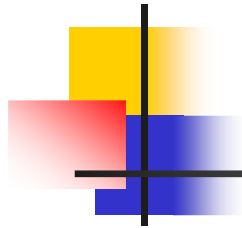
Пример class Point

```
public override bool Equals(Object obj) {  
    if (obj == null || this.GetType() != obj.GetType()) {  
        return false;  
    }  
    else {  
        Point p = (Point) obj;  
        return (x == p.x) && (y == p.y);  
    }  
}  
  
public override int GetHashCode() { return x ^ y; } }
```



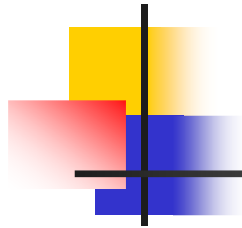
Пример class Point3D

```
class Point3D: Point {  
    int z;  
    public Point3D(int X, int Y, int Z) {  
        this.x = X;  
        this.y = Y;  
        this.z = Z;  
    }  
}
```



Пример class Point3D

```
public override bool Equals(Object obj) {  
    return base.Equals(obj) && z == ((Point3D)obj).z;  
}  
public override int GetHashCode() {  
    return base.GetHashCode() ^ z;  
}  
}
```



Перегрузка операторов

Общая форма перегрузки унарного оператора

```
public static возвр_тип operator op(тип_параметра операнд)
{...}
```

Общая форма перегрузки бинарного оператора

```
public static возвр_тип operator op(тип_парам1 операнд1,
    тип_парам2 операнд2)
{ ... }
```



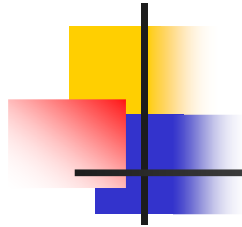

Возможности перегрузки

Операторы	Возможность перегрузки
+, -, !, ~, ++, --, true, false	Унарные можно
+, -, *, /, %, &, , ^, <<, >>	Бинарные можно
==, !=, <, >, <=, >=	Сравнения можно
&&,	Условные логические нельзя, но они вычисляются с помощью & и , допускающих перегрузку.
[]	Индексирования нельзя, но можно определить индексаторы.
()	Приведения нельзя, но можно определить операторы преобразования (explicit и implicit).



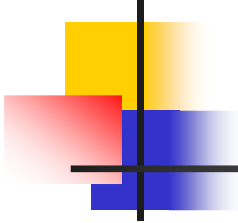
Возможности перегрузки

Операторы	Возможность перегрузки
<code>+=, -=, *=, /=, %=, &=, =, ^=, <<=, >>=</code>	Операторы присвоения нельзя, но, вычисляются с помощью бинарных, допускающих перегрузку.
<code>=, ., ?:, ->, new, is, sizeof, typeof</code>	Нельзя



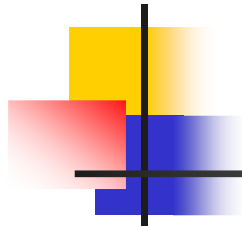
Перегрузка операторов + и -

```
public static ComplexNumber operator+ (ComplexNumber a,  
    ComplexNumber b) {  
    return new ComplexNumber(a.real + b.real, a.imaginary +  
        b.imaginary);  
}  
public static ComplexNumber operator- (ComplexNumber a,  
    ComplexNumber b) {  
    return new ComplexNumber(a.real - b.real, a.imaginary -  
        b.imaginary);  
}
```



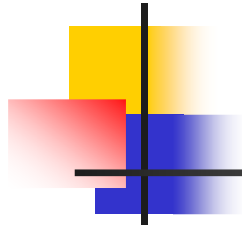
Перегрузка ToString()

```
public override string ToString() {  
    return(System.String.Format("{0} + {1}i", real, imaginary));  
}
```



Применение

```
ComplexNumber a = new ComplexNumber(10, 12);  
ComplexNumber b = new ComplexNumber(8, 9);  
Console.WriteLine("Complex Number a = {0}", a.ToString());  
Console.WriteLine("Complex Number b = {0}", b.ToString());  
ComplexNumber sum = a + b;  
Console.WriteLine("Complex Number sum = {0}", sum.ToString());  
ComplexNumber difference = a - b;  
Console.WriteLine("Complex Number difference = {0}",  
    difference.ToString());
```



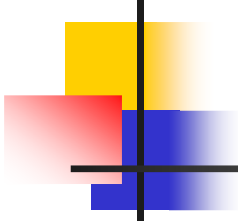
Результат

Complex Number $a = 10 + 12i$

Complex Number $b = 8 + 9i$

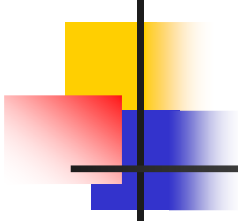
Complex Number sum $= 18 + 21i$

Complex Number difference $= 2 + 3i$



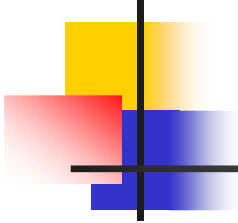
Перегружаем унарный оператор -

```
public static ComplexNumber operator -(ComplexNumber obj1) {  
    ComplexNumber arr = new ComplexNumber();  
    arr.a = -obj1.a;  
    arr.b = -obj1.b;  
    return arr;  
}
```



Перегружаем унарный оператор ++

```
public static MyPoint operator ++(Point obj1) {  
    obj1.x += 1;  
    obj1.y += 1;  
    obj1.z += 1;  
    return obj1;  
}
```

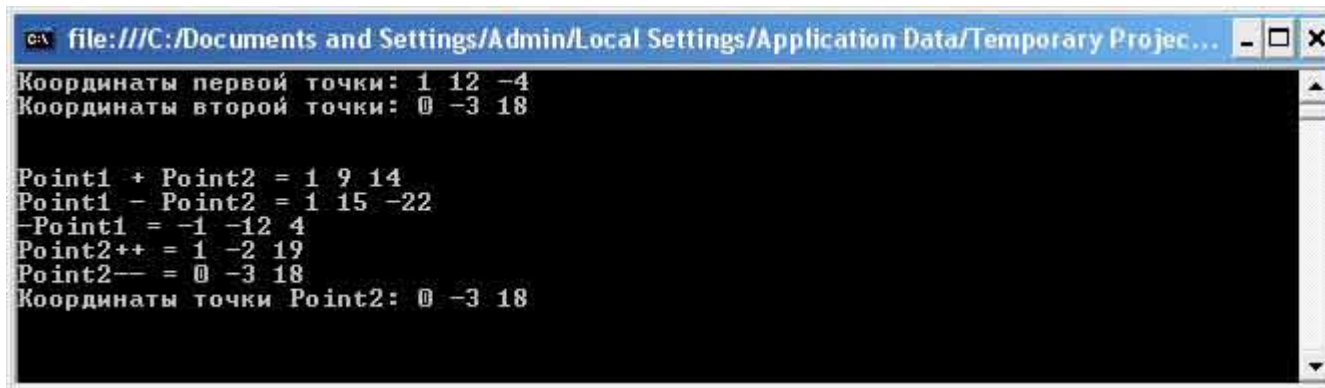



Перегрузка + для разных типов

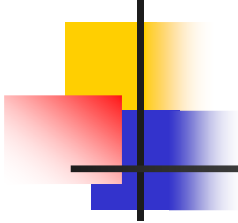
```
public static string operator +(MyArr obj1, string s) {  
    return s + " " + obj1.x + " " + obj1.y + " " + obj1.z;  
}
```

...

```
string s = Point2 + "Координаты точки Point2:";  
Console.WriteLine(s);
```

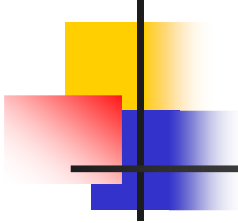


```
file:///C:/Documents and Settings/Admin/Local Settings/Application Data/Temporary Projec...  
Координаты первой точки: 1 12 -4  
Координаты второй точки: 0 -3 18  
  
Point1 + Point2 = 1 9 14  
Point1 - Point2 = 1 15 -22  
-Point1 = -1 -12 4  
Point2++ = 1 -2 19  
Point2-- = 0 -3 18  
Координаты точки Point2: 0 -3 18
```



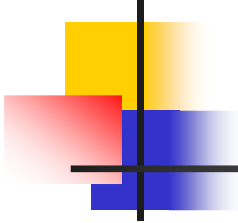
Перегрузка операторов == и !=

```
public struct Complex {  
    public double re, im;  
    public override bool Equals(Object obj) {  
        return obj is Complex && this == (Complex)obj;  
    }  
    public override int GetHashCode() {  
        return re.GetHashCode() ^ im.GetHashCode();  
    }  
    public static bool operator ==(Complex x, Complex y) {  
        return x.re == y.re && x.im == y.im; }  
    public static bool operator !=(Complex x, Complex y) {  
        return !(x == y);  
    }  
}
```



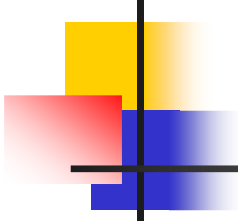
Определение неявного приведения implicit

```
class Digit {  
    public Digit(double d) { val = d; }  
    public double val;  
    // ...other members  
    // User-defined conversion from Digit to double  
    public static implicit operator double(Digit d) {  
        return d.val;  
    }  
    // User-defined conversion from double to Digit  
    public static implicit operator Digit(double d) {  
        return new Digit(d);  
    }  
}
```



Использование неявного приведения implicit

```
static void Main(string[] args) {  
    Digit dig = new Digit(7);  
    //This call invokes the implicit "double" operator  
    double num = dig;  
    //This call invokes the implicit "Digit" operator  
    Digit dig2 = 12;  
    Console.WriteLine("num = {0} dig2 = {1}", num,dig2.val);  
}
```



Определение явного приведения explicit

```
public static explicit operator Celsius(Fahrenheit f) {  
    return new Celsius((5.0f / 9.0f) * (f.degrees - 32));  
}
```

```
Fahrenheit f = new Fahrenheit(100.0f);  
Console.WriteLine("{0} fahrenheit", f.Degrees);  
Celsius c = (Celsius)f;
```